TECHNICAL PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

Developed Using Java and File Handling

---

1. Introduction

The Student Management System is a console-based application developed using the Java programming language. The primary objective of this project is to manage student information efficiently through a simple and user-friendly interface.

The system allows users to perform essential student management operations, including ***adding new students, viewing student records, searching for a particular student, updating student information, and deleting student records.***

The application uses file handling to store student information permanently. Unlike applications that store data only in temporary memory, this system saves student records in a text file, allowing the information to remain available even after the application is closed.

The project follows a layered software architecture by separating data models, business logic, repository operations, and user interaction.

2. Objectives of the Project

The major objectives of developing this system are:

1. To develop a simple and efficient student management application.
2. To implement **CRUD operations (Create, Read, Update, Delete).**
3. To understand object-oriented programming concepts in Java.
4. To implement file handling for permanent data storage.
5. To implement input validation to prevent invalid user entries.
6. To understand the Repository Pattern and separation of concerns.
7. To develop a modular application using Java packages and classes.
8. To provide an organized way of managing student academic information.

3. Technologies Used

Technology	Purpose
Java	Primary programming language
Java Collections Framework	Managing student records using ArrayList
Java I/O	Reading and writing student data
Scanner	Accepting user input
CSV-style text file	Persistent data storage
JDK	Compiling and executing the application

Development Environment

- Programming Language: Java
- Application Type: Console-based application
- Storage Mechanism: File-based storage
- Data Structure: ArrayList
- Architecture: Layered architecture

4. System Architecture

The application follows a layered architecture in which different components are responsible for different operations.

The project consists of the following layers:

4.1 Presentation Layer

Class: Main.java

The presentation layer provides interaction between the user and the application.

Responsibilities:

- Displaying the application menu.
- Accepting user choices.
- Calling appropriate methods from StudentManager.
- Managing the application execution flow.

4.2 Model Layer

Class: Student.java

The model layer represents the structure of student information.

It defines the attributes and methods associated with a student.

Attributes:

Attribute	Data Type	Description
id	int	Unique student identification number
name	String	Student's name
marks	double	Student's academic marks
grade	String	Student's academic grade

The Student class follows the principles of encapsulation by declaring its attributes as private and providing public getter and setter methods.

4.3 Service Layer

Class: StudentManager.java

The service layer contains the application's business logic.

Responsibilities:

- Adding student records.
- Displaying all student records.
- Searching for students using their IDs.
- Updating student information.
- Deleting individual student records.
- Deleting all student records.
- Checking student IDs to prevent duplicate entries.

The service layer communicates with the repository layer to save and retrieve student information.

4.4 Repository Layer

Class: StudentRepository.java

The repository layer handles persistent data storage.

It uses Java's File I/O classes to perform file operations.

Responsibilities:

- Saving student records into a file.
- Reading student records from a file.
- Converting file records into Student objects.
- Updating stored records.
- Removing all stored records.

The repository layer separates file-related operations from the application's business logic.

4.5 Utility Layer

Class: InputValidator.java

The utility layer provides reusable input validation methods.

Responsibilities:

- Validating integer inputs.
- Validating numeric marks.
- Ensuring marks remain within the range of 0 to 100.
- Preventing empty string inputs.
- Handling invalid user entries through repeated prompts.

5. Project Directory Structure

The project is organized into separate packages to maintain modularity and readability.

text

```
None
StudentManagementSystem/
│
├── src/
│   │
│   ├── app/
│   │   └── Main.java
│   │
│   ├── model/
│   │   └── Student.java
│   │
│   ├── service/
│   │   └── StudentManager.java
│   │
│   ├── repository/
│   │   └── StudentRepository.java
│   │
│   └── util/
│       └── InputValidator.java
│
├── out/
│
└── students.txt
```

The src directory contains the Java source files, while the out directory stores compiled class files.

The students.txt file stores student information permanently.

6. Functional Requirements

The system provides the following functionalities.

6.1 Add Student

The application allows users to add a new student by providing:

- Student ID
- Student Name
- Student Marks
- Student Grade

Before adding a student, the system validates the input and checks whether the entered ID already exists.

After successful validation, the student object is created and added to the ArrayList.

The updated student list is then saved to the file.

6.2 View Student List

The application displays all student records currently available in the system.

The displayed information includes student ID, name, marks, and grade.

The records are retrieved from the ArrayList maintained by the StudentManager.

6.3 Search Student

Users can search for a particular student using their unique student ID.

The system iterates through the ArrayList and compares the entered ID with the IDs of stored student objects.

If a matching record is found, the student's complete information is displayed.

Otherwise, the application displays an appropriate not-found message.

6.4 Update Student Details

The application allows users to modify the existing information of a student.

The student is identified using their ID.

Users can update:

- Name
- Marks
- Grade

After modification, the updated student list is saved to the file.

6.5 Delete Student

Users can delete an individual student record by entering the student's ID.

The application searches for the matching student and removes the corresponding object from the ArrayList.

The modified list is subsequently saved to the file.

6.6 Delete All Students

This functionality removes all student records from the application.

The ArrayList is cleared, and the contents of the storage file are also removed.

6.7 Exit Application

The application terminates when the user selects the exit option.

The previously saved student information remains stored in the file.

7. File Handling and Data Persistence

One of the important features of this project is permanent data storage using file handling.

The system uses a text file named students.txt to store student information.

7.1 Data Storage Format

Student information is stored using a comma-separated format.

Example:

text

```
None
101,Rahul,85.5,A
102,Priya,92.0,A+
103,Amit,76.0,B
```

Each line represents one student record.

The fields are arranged in the following order:

text

```
None
ID,Name,Marks,Grade
```

7.2 Writing Data

The application uses the following Java classes:

- FileWriter
- BufferedWriter

Whenever a student record is added, updated, or deleted, the repository rewrites the current student list into the file.

This ensures that the file contains the latest version of the student records.

7.3 Reading Data

The application uses:

- FileReader
- BufferedReader

When the application starts, the repository reads the existing file line by line.

Each line is divided into separate fields, and the values are used to construct Student objects.

These objects are then loaded into the ArrayList.

7.4 Data Persistence Workflow

1. The application starts.
2. StudentManager requests existing records from StudentRepository.
3. StudentRepository reads the data from students.txt.
4. The records are converted into Student objects.
5. The objects are stored in the ArrayList.
6. Users perform student management operations.
7. Whenever data changes, StudentRepository saves the updated list.
8. The updated records remain available after restarting the application.

8. Input Validation

Input validation ensures that users provide appropriate information while interacting with the application.

The project implements a separate InputValidator utility class.

The following validations are implemented:

Input	Validation
Student ID	Must be an integer
Student ID	Must be positive
Student ID	Must be unique
Student Name	Cannot be empty
Marks	Must be numeric
Marks	Must be between 0 and 100
Grade	Cannot be empty
Menu Choice	Must be an integer

The validation process prevents incorrect data from being accepted during normal application usage.

9. Object-Oriented Programming Concepts Used

The project demonstrates several important object-oriented programming concepts.

9.1 Encapsulation

The Student class uses private attributes to protect student information from direct external access.

Getter and setter methods provide controlled access to these attributes.

9.2 Abstraction

The StudentManager class provides methods such as addStudent(), deleteStudent(), and findStudent().

Users can perform operations without needing to understand the internal implementation of each operation.

9.3 Classes and Objects

The Student class acts as a blueprint, while individual student records are represented as objects created from that class.

9.4 Modularity

The application is divided into multiple packages and classes.

Each class has a specific responsibility, making the application easier to maintain, understand, and extend.

10. Data Structure Used

ArrayList

The project uses Java's ArrayList to maintain student records during application execution.

Declaration:

java

```
None
private ArrayList<Student> students;
```

Reasons for choosing ArrayList:

1. Dynamic resizing.
2. Easy insertion and deletion.
3. Convenient iteration.

4. Direct access to elements using indexes.
5. Supports storing Student objects.

The ArrayList acts as temporary in-memory storage, while the text file provides permanent storage.

11. Algorithm and Working Procedure

11.1 Application Startup

1. Start the application.
2. Initialize Scanner.
3. Create StudentManager.
4. Initialize StudentRepository.
5. Load existing student records from the file.
6. Display the main menu.

11.2 Adding a Student

1. Accept student ID.
2. Validate the ID.
3. Check whether the ID already exists.
4. Accept student name.
5. Accept student marks.
6. Validate marks.
7. Accept student grade.
8. Create a Student object.
9. Add the object to ArrayList.
10. Save the updated list to the file.

11.3 Searching for a Student

1. Accept student ID.
2. Iterate through the ArrayList.
3. Compare the entered ID with stored IDs.
4. Display the matching student information.
5. If no match exists, display an appropriate message.

11.4 Updating a Student

1. Accept student ID.
2. Search for the student.
3. If found, accept new information.
4. Update the corresponding object.
5. Save the updated ArrayList to the file.

11.5 Deleting a Student

1. Accept student ID.
2. Search for the student.
3. Remove the matching object.
4. Save the updated ArrayList to the file.
5. Display the result.

12. Complexity Analysis

Let N represent the total number of student records.

Operation	Time Complexity
Add Student	O(N) due to duplicate ID checking
View Student List	O(N)
Search Student	O(N)
Update Student	O(N)
Delete Student	O(N)
Delete All Students	O(N)
Load Students	O(N)
Save Students	O(N)

The operations involving searching and modification use linear traversal because the application currently uses an ArrayList.

The file reading and writing operations also require processing all student records.

13. Testing and Verification

The application should be tested using different input conditions to verify its functionality.

Test Case	Input/Action	Expected Result
Add student	Valid student details	Student added
Duplicate ID	Existing student ID	Entry rejected

Invalid ID	Alphabetic input	Input rejected
Empty name	Blank input	Input rejected
Invalid marks	Marks above 100	Input rejected
View records	Select option 2	Student list displayed
Search existing student	Valid ID	Student found
Search missing student	Nonexistent ID	Not-found message
Update student	Existing ID	Details updated
Delete student	Existing ID	Student removed
Delete missing student	Nonexistent ID	Not-found message
Delete all	Select option 6	All records removed
Restart application	Existing file	Previous records loaded

14. Advantages of the System

1. Simple and user-friendly console interface.
2. Permanent data storage using file handling.
3. Modular and organized source code.
4. Separation of business logic and data access.
5. Input validation to reduce invalid entries.
6. No external database required.
7. Easy to understand and maintain.
8. Demonstrates practical Java programming concepts.

15. Limitations of the System

Although the application provides essential student management functionality, it has certain limitations.

1. It supports only console-based interaction.
2. It uses a local text file instead of a database.
3. It does not support multiple users simultaneously.
4. It does not provide authentication or authorization.
5. Searching and updating records require linear traversal.
6. CSV-style storage requires additional handling for commas and special characters.
7. It does not provide advanced academic reports or analytics.
8. It does not currently provide a graphical user interface.

16. Future Enhancements

The system can be extended with several additional features.

16.1 Database Integration

Replace text-file storage with MySQL or PostgreSQL to support more reliable and scalable data management.

16.2 Graphical User Interface

Develop a graphical interface using Java Swing or JavaFX.

16.3 Student Authentication

Introduce login functionality for administrators and authorized users.

16.4 Sorting and Filtering

Allow users to sort students by name, marks, grade, or ID.

16.5 Academic Reports

Generate reports showing average marks, highest marks, lowest marks, and grade distributions.

16.6 Advanced Search

Implement searching by student name, grade, or marks.

16.7 Export Functionality

Allow student records to be exported into CSV or PDF formats.

16.8 Improved Data Storage

Implement robust CSV parsing, data backups, and transactional file updates to reduce the risk of data loss.

17. Conclusion

The Student Management System is a practical Java-based application developed to manage student information efficiently.

The project demonstrates the implementation of object-oriented programming, Java Collections, file handling, input validation, and layered architecture.

By using an ArrayList for managing student objects and a text file for permanent storage, the application provides a simple solution for performing basic student management operations.

The separation of responsibilities into model, service, repository, and utility packages makes the application modular, maintainable, and easier to extend.

Overall, this project provides a strong foundation for understanding how real-world software applications are designed and implemented using Java.

---

18. References

1. Oracle Java Documentation: https://docs.oracle.com/en/java/
2. Java Collections Framework: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/
3. Java I/O Documentation: https://docs.oracle.com/javase/8/docs/api/java/io/package-summary.html
4. Java Scanner Documentation: https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html
5. Java ArrayList Documentation: https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html

---

Project Title: Student Management System

Programming Language: Java

Application Type: Console-Based Application

Storage: File Handling (Text File)

Architecture: Layered Architecture

Project Status: Basic implementation with CRUD operations and persistent file storage.